

교과목 3 : 초거대언어모델(LLM)

단원 4 : 프롬프트 엔지니어링

DAY1. Vector DB

목 차

1. Vector DB 이해		3
2. Vector DB 역할		11
3. Vector DB 종류		20
4. 인덱스 영속화 아키텍처		28
5. Vector DB를 활용한 검색		41

1. Vector DB 이해

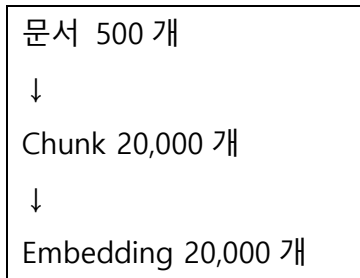
대량 검색과 인덱스 저장

예를 들어 회사에는

- 제품 설명서(PDF)
- 업무 매뉴얼
- 사내 규정
- 환불 정책
- 고객 FAQ
- 기술 문서
- 계약서

등이 수백 개에서 수천 개까지 존재합니다.

이 문서들을 잘게 나누면(Chunking)



처럼 엄청난 양의 벡터가 만들어집니다.

이렇게 되면 단순한 NumPy 검색으로는 처리하기 어려워집니다.

그래서 사용하는 것이 **Vector Database** 입니다.

제공된 샘플 FAQ 가 10 개밖에 없었기 때문에 NumPy 만으로도 충분했습니다.

그러나 회사 문서가 많아지면

- 전체 비교가 너무 느려지고
- 매번 임베딩을 다시 하면 API 비용이 매우 많이 발생합니다.

이를 해결하기 위해 사용하는 것이 **벡터 데이터베이스(Vector Database)** 입니다.

즉, 많은 문서를 빠르게 검색하기 위한 전용 데이터베이스가 Vector DB 입니다.

1. NumPy 검색의 한계

LLM FAQ 검색을 다음과 같이 진행합니다.

질문 입력

↓

질문 임베딩

↓

FAQ 벡터들과 모두 비교

↓

가장 비슷한 문장 선택

FAQ 가 10 개뿐이라면 10 번 비교하면 끝입니다.

CPU 입장에서는 거의 순식간에 끝나는 작업입니다.

그런데 실제 회사는 다릅니다.

예를 들어 쇼핑몰 회사라고 가정해 보겠습니다.

문서가 다음처럼 존재합니다.

- 환불 정책
- 배송 정책
- 회원 등급 정책
- 제품 설명서
- 사용 매뉴얼
- 회사 규정
- 기술 문서
- 고객 FAQ
- ...

이런 문서를 모두 Chunking 하면

수천 개

↓

수만 개

↓

수십만 개

의 Chunk 가 생성됩니다.

첫 번째 문제 : 속도

NumPy 검색은

질문

↓

모든 벡터와 비교

↓

가장 큰 값 선택

이라는 구조입니다.

즉,

벡터 10 개

↓

10 번 계산

이면 금방 끝납니다.

하지만

벡터 100,000 개

↓

100,000 번 계산

해야 합니다.

질문이 들어올 때마다

10 만 번

20 만 번

50 만 번

계산하게 됩니다.

사용자가

"배송은 언제 오나요?"

라고 질문했는데

검색 시간이

5 초

8 초

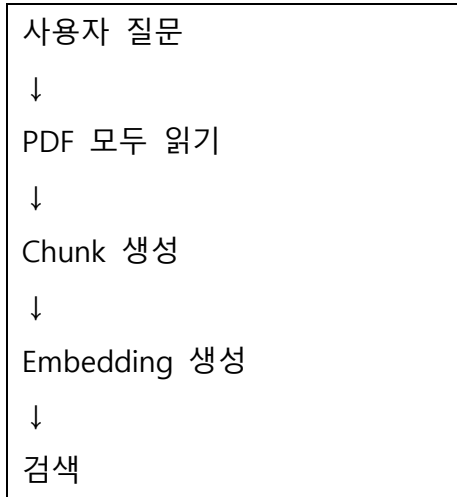
10 초

걸린다면 서비스로 사용할 수 없습니다.

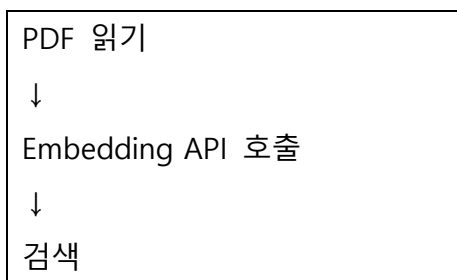
두 번째 문제 : 재인덱싱

더 큰 문제는 **매 요청마다 문서를 다시 임베딩하는** 경우입니다.

예를 들어 잘못 작성된 프로그램은 다음과 같습니다.



즉, 질문 하나 들어올 때마다

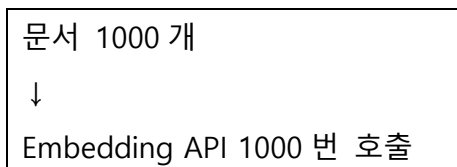


을 반복합니다.

왜 문제가 될까요?

Embedding 은 무료 계산이 아닙니다.

OpenAI API 를 사용하는 경우



이 일어납니다.

질문이 들어올 때마다 이것을 반복하면 **API 비용 증가 + 응답 시간 증가** 라는 문제가 동시에 발생합니다.

즉, 돈도 많이 들고 속도도 느려집니다.

예제 코드 설명

```
# 끔찍한 코드 — 요청마다 전체 재임베딩!  
def handle_request(question):  
    chunks = load_and_chunk_all_pdfs()    # 매번 PDF 로드  
    vectors = embed(chunks)               # 매번 임베딩 (API 비용!)  
    return search(vectors, question)      # 그제서야 검색
```

이 코드는 매우 비효율적인 예입니다.

동작 과정은 다음과 같습니다.

```
질문  
↓  
PDF 읽기  
↓  
Chunk 생성  
↓  
Embedding 생성  
↓  
검색
```

즉, 질문이 들어올 때마다 모든 문서를 처음부터 다시 처리합니다.

실제 서비스에서는 절대로 이렇게 구현하지 않습니다.

2. 벡터 DB 가 해결하는 것

Vector Database 는 위의 두 가지 문제를 모두 해결합니다.

해결 방법은 매우 간단합니다.

문서를 한 번만 임베딩하여 저장해 두는 것입니다.

즉,

```
처음 한 번  
↓  
문서 읽기  
↓  
Chunk 생성  
↓
```

Embedding
↓
Vector DB 저장

이후에는

질문
↓
질문만 임베딩
↓
Vector DB 검색

만 수행합니다.

문서를 다시 읽지 않습니다. 문서를 다시 임베딩하지도 않습니다.

첫 번째 해결 방법

인덱스를 미리 만들어 둔다.

PDF
↓
Chunk
↓
Embedding
↓
Index 생성
↓
Disk 저장

이 과정을 한 번만 수행합니다. 이를 **영속화(Persistence)** 라고 합니다.

즉, 프로그램을 종료해도 디스크에 저장되어 있으므로 다시 만들 필요가 없습니다.

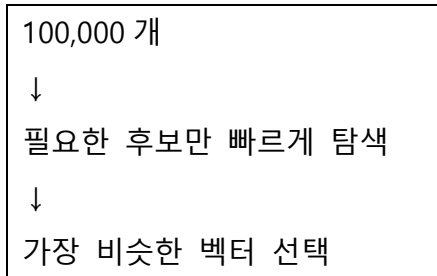
두 번째 해결 방법

ANN 검색

Vector DB 는 모든 벡터를 비교하지 않습니다.

대신 ANN(Approximate Nearest Neighbor) 알고리즘을 사용합니다.

즉,



이렇게 하기 때문에 매우 빠릅니다.

[사전 1회] PDF → 토크킹 → 임베딩 → 벡터DB 인덱스 → 디스크 저장
[매 요청] 디스크에서 로드 → 질문만 임베딩 → 빠르게 검색

즉, 문서는 한 번만 계산합니다. 질문만 새로 계산합니다.

핵심 원리

비싼 계산은 미리 수행하고 저장해 두며, 실제 요청 시에는 저장된 결과만 빠르게 조회한다.

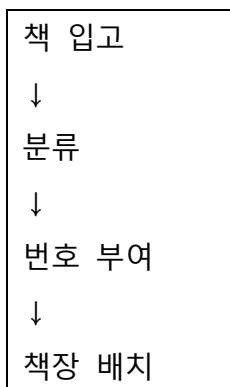
이것이 인덱싱(Indexing)의 핵심 원리입니다.

예를 들어, 데이터베이스에서 검색 속도를 높이기 위해 인덱스를 미리 만들어 두는 것과 같은 개념입니다.

도서관 비유

도서관에는 수만 권의 책이 있습니다.

사서는



를 한 번만 수행합니다.

학생이 "파이썬 책 주세요." 라고 요청하면
사서는 책을 다시 분류하지 않습니다.
이미 만들어 놓은 분류 체계를 이용하여 바로 책 위치를 찾습니다.

Vector DB 도 완전히 같은 원리입니다.
문서를 미리 분류(인덱싱)해 두고,
검색 시에는 그 인덱스를 이용하여 빠르게 원하는 정보를 찾습니다.

FAISS 사용

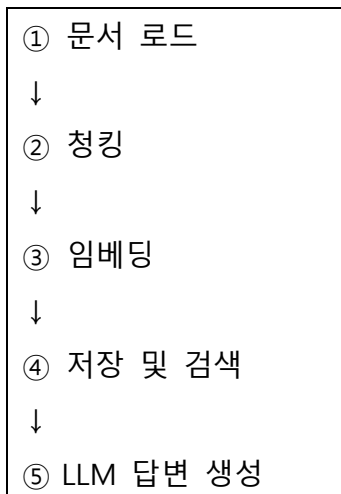
이번 수업에서는 **FAISS** 를 중심으로 실습합니다.
그 이유는

- 매우 빠르고
- 가볍고
- 파일 저장이 쉽고
- 대규모 검색 성능이 뛰어나기 때문입니다.

반면, 메타데이터를 함께 관리하거나 다양한 필터링 기능이 필요한 경우에는 **Chroma** 를 사용할 수 있습니다.

3. RAG 5 단계 중 저장/검색 단계

RAG 는 일반적으로 다음의 5 단계로 구성됩니다.



2. Vector DB 역할

많은 사람들이 벡터 DB 를 단순히 "벡터를 저장하는 데이터베이스" 정도로 생각하지만, 실제 핵심 역할은 그것이 아닙니다.

벡터 DB 의 가장 중요한 역할은 다음 두 가지 작업을 **완전히 분리하는** 것입니다.

- 문서를 미리 처리하여 저장하는 작업(인덱싱)
- 사용자의 질문에 빠르게 답하는 작업(서비스)

이 두 작업을 분리하기 때문에 RAG(Retrieval-Augmented Generation)가 실제 서비스에서 빠르고 효율적으로 동작할 수 있습니다.

벡터 DB 의 핵심은 **인덱싱(사전 1 회)**과 **서비스(매 요청)**를 분리하는 것입니다.

즉,

- 인덱싱은 시간이 오래 걸리고 비용이 많이 드는 작업이지만 **한 번만 수행**합니다.
- 검색은 매우 빠르고 비용이 적기 때문에 **사용자가 질문할 때마다 반복 수행**합니다.

이러한 구조 덕분에 RAG 시스템을 실제 서비스에서 사용할 수 있습니다.

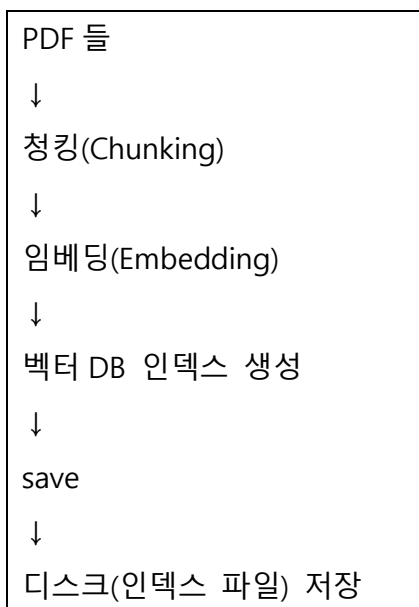
1. 두 시점의 분리

벡터 DB 는 모든 작업을 한 번에 수행하지 않습니다.

작업을 다음 두 시점으로 나누어 처리합니다.

첫 번째 단계 : 사전 인덱싱

이 작업은 문서를 미리 준비하는 과정입니다.

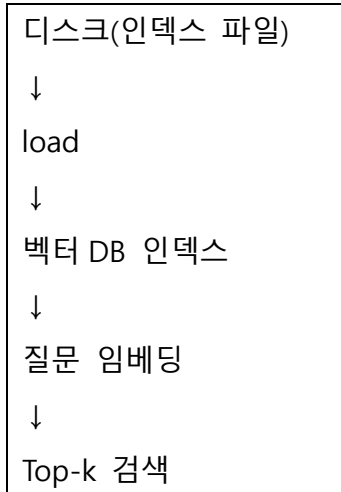


이 과정은 **한 번만 수행**합니다.

왜냐하면 문서는 자주 변경되지 않기 때문입니다.

두 번째 단계 : 서비스

사용자가 질문을 할 때 수행되는 과정입니다.



여기서는 문서를 다시 처리하지 않습니다.

이미 만들어진 인덱스를 이용하여 빠르게 검색만 수행합니다.

왜 두 작업을 나누는가?

인덱싱(Indexing) 과 **질의(Query)** 를 분리하는 개념을 배웠습니다.

벡터 DB에서는 이 개념을 실제 프로그램으로 구현한 것입니다.

즉,

- 인덱싱은 미리 수행
- 질의는 필요할 때마다 수행

이라는 구조입니다.

2. 인덱싱 — 무겁지만 한 번만

인덱싱이란 문서를 검색 가능한 형태로 변환하여 저장하는 작업입니다.

예제 코드는 다음과 같습니다.

```
# 인덱싱: 느리고 비싸지만 한 번만
chunks = load_and_chunk(pdfes)          # 로드 + 청킹
vs = FAISS.from_documents(chunks, emb)   # 임베딩 + 인덱스 생성 (여기서 비용 발생!)
vs.save_local("faiss_index")             # 디스크에 저장
```

코드 동작 과정

① 문서를 읽습니다.

PDF

↓

로드

먼저 PDF 파일을 메모리로 불러옵니다.

② 청킹합니다.

문서

↓

Chunk 1

Chunk 2

Chunk 3

...

긴 문서를 작은 조각으로 나누어 검색하기 쉽게 만듭니다.

③ 임베딩합니다.

각 Chunk 를 숫자 벡터로 변환합니다.

Chunk

↓

Embedding Vector

이 과정에서 OpenAI Embedding API 와 같은 모델을 호출하기 때문에 시간도 걸리고 비용도 발생합니다.

④ 벡터 인덱스를 생성합니다.

생성된 벡터들을 FAISS 내부 자료구조에 저장합니다.

이 자료구조가 바로 **검색 인덱스(Index)** 입니다.

⑤ 디스크에 저장합니다.

FAISS Index

↓

save_local()

↓

faiss_index 폴더

프로그램을 종료해도 다음에 다시 사용할 수 있도록 저장합니다.

왜 한 번만 수행할까요?

이 과정은

- 임베딩 API 호출
- 벡터 생성
- 인덱스 생성

등 많은 계산이 필요합니다.

따라서

- 시간도 오래 걸리고
- API 비용도 발생합니다.

하지만 문서가 변경되지 않는다면 다시 만들 필요가 없습니다.

3. 서비스 — 가볍고 매번

서비스는 이미 저장된 인덱스를 사용하는 단계입니다.

예제 코드입니다.

```
# 서비스: 빠르고 싸서 매 요청마다
vs = FAISS.load_local("faiss_index", emb, ...) # 저장된 인덱스 로드 (재임베딩 X!)
results = vs.similarity_search("질문", k=3)      # 빠른 검색
```

코드 동작 과정

① 저장된 인덱스를 불러옵니다.

Disk

↓

load_local()

↓

FAISS Index

여기서 중요한 점은 **문서를 다시 임베딩하지 않는다는** 것입니다.

이미 저장되어 있는 벡터를 그대로 읽기만 합니다.

② 질문만 임베딩합니다.

예를 들어

"배송은 언제 오나요?"

라는 질문이 들어오면 질문 문장 하나만 임베딩합니다.

③ 검색합니다.

질문 벡터

↓

FAISS 검색

↓

Top-k 결과

유사한 문서를 빠르게 찾아줍니다.

load_local의 핵심

load_local()

↓

재임베딩하지 않음

↓

저장된 벡터 그대로 사용

즉, 문서를 다시 계산하지 않습니다.

그래서 속도가 매우 빠르고 비용도 거의 발생하지 않습니다.

4. 근사 최근접 이웃(ANN) — 빠른 검색의 비결

벡터 DB 가 빠른 이유는 ANN(Approximate Nearest Neighbor, 근사 최근접 이웃) 알고리즘을 사용하기 때문입니다.

일반적인 검색

벡터 100,000 개

↓

100,000 개 모두 비교

↓

가장 가까운 벡터 선택

정확하지만 시간이 오래 걸립니다.

ANN 검색

벡터 100,000 개

↓

가까울 가능성이 높은 후보만 선택

↓

후보끼리 비교

↓

가장 가까운 벡터 선택

즉, 전체를 비교하지 않고 가능성이 높은 후보만 빠르게 탐색합니다.

정확도는?

정확한 검색

↓

모든 벡터 비교

↓

정확하지만 느림

반면

ANN 검색

↓

가까운 후보만 검색

↓

약간의 오차

↓

매우 빠름

즉, 아주 약간의 정확도를 양보하는 대신 엄청난 속도를 얻습니다.

그래서 수만 개, 수십만 개, 수백만 개의 벡터도 밀리초(ms) 단위로 검색할 수 있습니다.

예를 들어 "가방" 이라는 단어를 찾는다고 생각해 보겠습니다.

NumPy 방식은

첫 페이지

↓

둘째 페이지

↓

셋째 페이지

↓

...

↓

끝 페이지

까지 모두 확인합니다.

반면 ANN 은

ㄱ

↓

"가" 부근

↓

가방

처럼 처음부터 가능성이 높은 위치로 이동합니다.

따라서 훨씬 빠르게 찾을 수 있습니다.

5. 왜 이 분리가 중요한가

잘못된 방법

```
# 분리 안 함 (끔찍) — 요청마다 인덱싱
def handle(question):
    vs = FAISS.from_documents(load_and_chunk(pdf), emb) # 매번 임베딩!
    return vs.similarity_search(question)
```

이 코드는 질문이 들어올 때마다

```
PDF 읽기
↓
청킹
↓
임베딩
↓
인덱스 생성
↓
검색
```

를 반복합니다. 즉, 매 요청마다 비용이 많이 발생합니다.

올바른 방법

```
# 분리함 (올바름) — 인덱싱 1 회, 검색 매번
vs = FAISS.load_local("faiss_index", emb, ...) # 서버 기동 시 1 회
def handle(question):
    return vs.similarity_search(question) # 검색만
```

이 방식에서는 서버가 시작될 때 인덱스 로드를 한 번만 수행합니다.

그 이후에는

```
질문
↓
검색
↓
결과 반환
```

만 반복합니다.

이렇게 하면 속도도 빠르고 API 비용도 크게 줄어듭니다.

핵심 정리

이번 챕터에서 반드시 이해해야 할 내용은 다음과 같습니다.

1. **벡터 DB 는 인덱싱(사전 1 회)과 서비스(매 요청)를 분리하여 처리합니다.**
2. 인덱싱은 문서를 로드하고, 청킹하고, 임베딩한 뒤 인덱스를 생성하여 저장하는 과정으로 **시간과 비용이 많이 들지만 한 번만 수행합니다.**
3. 서비스 단계에서는 `load_local()`을 사용하여 **저장된 인덱스를 그대로 불러오기 때문에 문서를 다시 임베딩하지 않습니다.**
4. 벡터 DB 는 **ANN(Approximate Nearest Neighbor)** 알고리즘을 사용하여 수만 개 이상의 벡터에서도 매우 빠르게 유사도 검색을 수행합니다.
5. 이러한 **인덱싱과 서비스의 분리 구조가 RAG 를 실제 서비스 환경에서 빠르고 경제적으로 운영할 수 있게 만드는 핵심 원리입니다.**

3. Vector DB 종류

벡터 데이터베이스(Vector Database)는 텍스트, 이미지, 음성 등의 데이터를 벡터(Embedding) 형태로 저장하고 유사도 검색(Similarity Search)을 수행하는 데이터베이스입니다.

최근 LLM(RAG), 생성형 AI, 추천 시스템, 이미지 검색 등의 핵심 기술로 사용되고 있습니다.

벡터 DB 분류

대표적인 벡터 DB는 다음과 같습니다.

벡터DB	특징	오픈소스	클라우드	메타데이터 검색	추천 용도
FAISS	매우 빠른 벡터 검색 라이브러리	O	X	제한적	학습, 단일 서버
Chroma	LangChain과 연동이 쉬움	O	X	O	RAG 개발
Milvus	대용량 분산 벡터DB	O	O	O	기업 서비스
Qdrant	Rust 기반 고성능 DB	O	O	매우 우수	실무 RAG
Pinecone	완전 관리형 클라우드	X	O	O	SaaS 서비스
Weaviate	Graph + Vector 검색	O	O	매우 우수	AI 검색
Elasticsearch	벡터 + 키워드 검색	O	O	매우 우수	검색 시스템
OpenSearch	Elasticsearch 기반	O	O	매우 우수	AWS 환경
LanceDB	AI 전용 로컬 DB	O	일부	O	로컬 AI 개발

어떤 벡터 DB를 선택해야 할까?

사용 목적	추천 벡터DB	이유
AI 및 RAG 학습	FAISS	설치와 사용이 가장 쉽고 검색 속도가 빠릅니다.
PDF 기반 챗봇	Chroma	LangChain과의 연동이 쉽고 메타데이터 관리가 편리합니다.
기업용 RAG 서비스	Qdrant	성능과 메타데이터 필터링이 우수하며 운영이 용이합니다.
초대규모 데이터 (수억~수십억 벡터)	Milvus	분산 처리와 대규모 확장성이 뛰어납니다.
서버 관리 없이 빠른 구축	Pinecone	완전 관리형 클라우드 서비스로 인프라 운영 부담이 적습니다.

사용 목적	추천 벡터DB	이유
키워드 + 의미 검색 통합	Elasticsearch/OpenSearch	기존 전문 검색과 벡터 검색을 함께 사용할 수 있습니다.

FAISS vs Chroma

이 챕터에서는 실제 RAG 시스템에서 가장 많이 사용하는 두 가지 벡터 저장소(Vector Store)인 **FAISS**와 **Chroma**를 비교합니다.

두 프로그램 모두 벡터를 저장하고 유사도 검색을 수행하는 역할을 하지만, 목적과 장점이 서로 다릅니다. **Chroma**는 메타데이터 필터와 컬렉션 관리 기능이 강한 임베디드 데이터베이스입니다. 따라서

- 일반적인 RAG에서는 **FAISS**를 주력으로 사용하고,
- 특정 조건으로 문서를 검색해야 하는 경우에는 **Chroma**를 사용합니다.

1. 두 벡터스토어 비교

항목	FAISS	Chroma
형태	라이브러리(인메모리 인덱스)	임베디드 DB(메타데이터·필터 강함)
영속화	save_local() / load_local()	persist_directory
장점	매우 빠르고 가벼움	메타데이터 필터와 컬렉션 관리가 편리
추천 상황	읽기 위주, 단순, 고속 검색	문서 갱신, 메타데이터 필터, 여러 컬렉션 관리

두 프로그램 모두

문서

↓

Embedding

↓

벡터 저장

↓

유사도 검색

을 수행합니다.

그러나 저장 방식과 관리 기능이 다릅니다.

2. FAISS — 가볍고 빠른 라이브러리

FAISS는 **Facebook AI Similarity Search** 의 약자입니다.
Meta(Facebook)에서 개발한 벡터 검색 라이브러리입니다.

```
from langchain_community.vectorstores import FAISS

vs = FAISS.from_documents(chunks, emb)  # 인덱스 생성
vs.save_local("faiss_index")           # 파일로 저장
vs2 = FAISS.load_local(
    "faiss_index",
    emb,
    allow_dangerous_deserialization=True
) # 로드
```

코드 동작 과정

① 인덱스 생성

문서

↓

Chunk

↓

Embedding

↓

FAISS Index 생성

문서를 벡터로 변환하여 FAISS 인덱스를 생성합니다.

② 파일 저장

FAISS Index

↓

save_local()

↓

faiss_index

생성한 인덱스를 파일 형태로 저장합니다.

③ 다시 불러오기

faiss_index

↓

load_local()

↓

메모리

프로그램을 다시 실행하더라도 저장한 인덱스를 그대로 사용할 수 있습니다.

FAISS의 장점

- **매우 빠르다.** : 대량의 벡터 검색에 최적화되어 있습니다.
- **가볍다.** : 의존성이 적어서 설치와 배포가 쉽습니다.
- **영속화가 명확하다.** : 다음 두 함수만 기억하면 됩니다.

언제 사용하는가?

- 문서가 자주 변경되지 않는 경우
- 빠른 검색이 중요한 경우
- 복잡한 메타데이터 필터가 필요하지 않은 경우

대부분의 RAG 시스템은 FAISS만으로도 충분합니다.

3. Chroma — 메타데이터에 강한 DB

Chroma는 메타데이터 관리 기능이 강한 임베디드 벡터 데이터베이스입니다.

```
from langchain_community.vectorstores import Chroma

vs = Chroma.from_documents(chunks, emb, persist_directory="chroma_db")
results = vs.similarity_search(
    "질문",
    k=3,
    filter={"source": "멤버십정책.pdf"}
)
```

코드 동작 과정

① 벡터 저장

문서

↓

Embedding

↓

Chroma DB

생성된 벡터를 자동으로 저장합니다.

② 검색

질문

↓

유사도 검색

여기까지는 FAISS와 거의 같습니다.

③ 메타데이터 필터

```
filter={"source":"멤버십정책.pdf"}
```

이 의미는

모든 문서 검색

↓

아님

↓

멤버십정책.pdf 안에서만 검색입니다.

즉, 원하는 문서만 검색할 수 있습니다.

Chroma의 장점

메타데이터 필터

- 특정 문서
- 특정 날짜
- 특정 카테고리

등으로 검색 범위를 제한할 수 있습니다.

자동 영속화

persist_directory

만 지정하면 자동으로 저장됩니다.

따라서 직접 save()를 호출할 필요가 없습니다.

컬렉션 관리

여러 종류의 문서를 따로 관리할 수 있습니다.

예를 들어

- 제품 매뉴얼
- FAQ
- 회사 규정
- 회원 정책

을 각각 별도의 컬렉션으로 관리할 수 있습니다.

언제 사용하는가?

- 특정 문서 안에서만 검색
- 최신 문서만 검색
- 정책 문서만 검색

처럼 메타데이터를 이용한 필터링 검색이 필요한 경우입니다.

4. 무엇을 언제 쓰나

메타데이터 필터가 필요한가?

예를 들어

- 특정 문서만
- 특정 날짜만
- 특정 카테고리만

검색해야 합니까?

아니오 → FAISS

가볍고 빠르기 때문에 대부분의 경우 충분합니다.

예 → Chroma

메타데이터 필터 기능이 강력하므로 필터링 검색이 가능합니다.

대부분은 FAISS로 충분

정책 문서 전체에서 검색이라면 FAISS가 가장 간단합니다.

반대로 멤버십 정책 안에서만 검색처럼 조건을 제한해야 하는 경우에는 Chroma를 사용합니다.

5. similarity_search의 점수

검색 점수를 해석할 때 주의해야 할 사항입니다.

```
scored = vs.similarity_search_with_score(query, k=2)

for d, score in scored:
    print(
        f"거리 {score:.3f} | {d.page_content[:50]}"
    )
```

중요

FAISS 점수는 "거리"라서 작을수록 가깝습니다(코사인 유사도와 반대 방향!)

즉, 점수가 작다

↓

더 가깝다 입니다.

코사인 유사도와 차이

코사인 유사도는 값이 클수록 가깝다

하지만 FAISS에서는 거리(distance) 값이 작을수록 가깝다 입니다.

즉, 완전히 반대입니다.

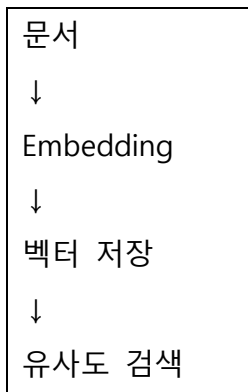
그래서 검색 결과를 해석할 때 반드시 사용하는 벡터스토어의 점수 의미를 확인해야 합니다.

6. 더 큰 벡터DB들

Pinecone, Weaviate, Milvus, pgvector 등. 이들은 분산·대규모·클라우드 환경을 위한 것입니다.

소규모 · 로컬 → FAISS, Chroma (이 과정)
대규모 · 클라우드 → Pinecone, Weaviate, Milvus, pgvector

어떤 벡터DB를 사용하든 원리는 동일합니다.



차이는

- 저장 규모
- 운영 방식
- 관리 기능

뿐입니다.

핵심 정리

이번 챕터에서 반드시 기억해야 할 내용은 다음과 같습니다.

1. **FAISS**는 Meta에서 개발한 가볍고 빠른 벡터 검색 라이브러리로, `save_local()`과 `load_local()`을 이용해 인덱스를 저장하고 불러올 수 있으며 대부분의 RAG 시스템에서 충분한 성능을 제공합니다.
2. **Chroma**는 메타데이터 필터링, 자동 영속화(`persist_directory`), 컬렉션 관리 기능을 제공하는 임베디드 벡터 데이터베이스로, 특정 문서나 조건에 한정된 검색이 필요한 경우에 적합합니다.
3. 대부분의 일반적인 RAG에서는 **FAISS만으로도 충분하며, 메타데이터 기반의 정밀한 검색이 필요할 때 Chroma를 선택합니다.**
4. **FAISS의 `similarity_search_with_score()`가 반환하는 점수는 거리(distance) 이므로 값이 작을수록 더 유사한 문서를 의미합니다.** 이는 코사인 유사도와 반대 개념이므로 혼동하지 않아야 합니다.
5. 대규모 클라우드 환경에서는 Pinecone, Weaviate, Milvus, pgvector 와 같은 벡터 DB 를 사용할 수 있지만, **문서를 임베딩하여 저장하고 유사도로 검색한다는 기본 원리는 모두 동일합니다.**

4. 인덱스 영속화 아키텍처

가장 흔한 초보 실수는 요청 핸들러 안에서 `FAISS.from_documents()`를 호출하는 것입니다.

이렇게 하면 사용자가 질문할 때마다

- 문서를 다시 읽고,
- 다시 청킹하고,
- 다시 임베딩하고,
- 다시 인덱스를 생성하게 됩니다.

결국 매 요청마다 전체 임베딩이 반복되어 속도도 매우 느려지고 API 비용도 크게 증가합니다.

따라서 반드시

- 인덱싱(배치 작업)
- 서비스(검색 작업)

를 서로 다른 파일과 단계로 분리해야 합니다.

1. 치명적인 초보 실수

벡터 DB 를 처음 사용할 때 가장 많이 하는 실수입니다.

다음은 **절대로 사용하면 안 되는 코드**입니다.

```
# ❌ 절대 하면 안 되는 코드
def handle_request(question):
    docs = load_all_pdfs()           # 요청마다 PDF 로드
    chunks = split(docs)            # 요청마다 청킹
    vs = FAISS.from_documents(chunks, emb) # 요청마다 전체 임베딩! (비용 폭발)
    return vs.similarity_search(question)
```

이 코드는 요청이 올 때마다 전체 문서를 다시 임베딩합니다. 질문 하나당:

- 수천 개 청크 임베딩 → API 비용 막대
- 몇 초~몇십 초 지연 → 사용자 이탈

문서가 안 바뀌었는데도 매번 처음부터 다시 만드는 거죠. 도서관에서 손님이 올 때마다 책을 전부 다시 분류하는 셈입니다.

2. 올바른 분리 — 두 개의 파일

해결책은 인덱싱과 서비스를 별개의 파일/단계로 분리하는 것입니다.

```
■ build_index.py — 배치/배포 시 1회 실행
PDF 로드 → 청킹 → 임베딩 → FAISS.from_documents → save_local("faiss_index")
(느리고 비싸지만 한 번만)

■ service.py — 서버 기동 시 1회 load, 요청마다 검색
load_local("faiss_index") ← 서버 시작할 때 한 번
def handle(question):
    return vs.similarity_search(question) ← 요청마다 검색만
```

핵심은 from_documents(인덱싱)는 build_index.py 에, load_local(로드)는 service.py 에 두는 것입니다. 이렇게 분리하면 임베딩은 배포 때 한 번만, 서비스는 검색만 합니다.

3. 백엔드 패턴과 같다

이 분리는 백엔드 개발자에게 익숙한 패턴입니다.

벡터DB	백엔드 대응
인덱싱(build_index.py)	DB 마이그레이션 / 시드 데이터
서비스(service.py)	API 핸들러

DB 마이그레이션을 매 요청마다 돌리지 않죠. 배포할 때 한 번 하고, API 는 조회만 합니다. 벡터 DB 도 똑같습니다 — 인덱싱은 배포 시 1 회, 검색은 요청마다.

DB Migration

데이터베이스를 만들 때 매 요청마다 CREATE TABLE 을 실행하지 않습니다. 배포할 때 한 번만 실행합니다.

API Handler

사용자가 요청하면 조회 수정 삭제 등 필요한 작업만 수행합니다.

4. 서버 기동 시 1회 로드

서비스 코드에서는 인덱스를 모듈 로드 시(서버 기동 시) 한 번 읽어 메모리에 둡니다.

```
# service.py
from langchain_community.vectorstores import FAISS
from common import get_embeddings

# 서버 기동 시 1 회만 실행 — 인덱스를 메모리에 상주시킴
_vs = FAISS.load_local(
    "faiss_index",
    get_embeddings(),
    allow_dangerous_deserialization=True
)

def search(question, k=3):
    # 요청마다 이 함수만 호출 — 검색만 (재로드 X)
    return _vs.similarity_search(question, k=k)
```

_vs 는 모듈 전역으로 한 번 로드되어 메모리에 상주합니다. 그래서 요청마다 search 만 부르면 빠른 검색이 됩니다. 11:12 강에서 "데이터는 모듈 로드 시 1 회만 읽는다"고 한 그 원칙과 같죠.

5. 인덱스가 바뀌면?

문서가 추가/수정되면 인덱스를 갱신해야 합니다. 두 가지 방법이 있습니다.

- **전체 재빌드**: build_index.py 를 다시 실행 (간단하지만 전체 재임베딩)
- **증분 업데이트**: 새 문서만 add_documents 로 추가 (08 번에서 배움)

문서가 가끔 바뀌면 전체 재빌드가 간단합니다. 자주 바뀌거나 문서가 많으면 증분 업데이트가 효율적이죠. 어느 쪽이든, **서비스는 갱신된 인덱스를 다시 로드하면 됩니다.**

6. 영속화 = 아키텍처 설계

"인덱스를 어떻게 저장하고 로드하는가"는 단순한 코드 문제가 아니라 **시스템 설계(아키텍처)** 입니다.

- 인덱싱과 서비스를 분리하면 → 빠르고 싸고 안정적
- 분리하지 않으면 → 느리고 비싸고 서비스 불가

그래서 "인덱스 영속화가 곧 아키텍처"입니다. RAG 를 실제 서비스로 만들려면, 이 분리를 처음부터 염두에 두어야 합니다.

핵심 정리

이번 내용에서 반드시 기억해야 할 내용은 다음과 같습니다.

1. 가장 흔한 초보자의 실수는 **요청 핸들러 안에서 FAISS.from_documents()를 호출하여 매 요청마다 문서를 다시 임베딩하는 것**입니다.
2. 올바른 구조는 **build_index.py** 에서 인덱스를 생성하고 저장한 뒤, **service.py** 에서는 **load_local()**로 인덱스를 한 번만 불러와 검색만 수행하는 것입니다.
3. 이 구조는 백엔드의 **DB 마이그레이션과 API 핸들러를 분리하는 방식**과 동일한 설계 원리를 따릅니다.
4. 서비스에서는 **서버가 시작될 때 인덱스를 한 번만 메모리에 로드**하고, 이후 모든 사용자 요청은 메모리에 있는 인덱스를 이용하여 빠르게 검색합니다.
5. 문서가 변경되면 **전체 재빌드(build_index.py 실행)** 또는 **증분 업데이트(add_documents)** 방식으로 인덱스를 갱신할 수 있습니다.
6. **인덱스 영속화는 단순한 저장 기능이 아니라 RAG 시스템 전체의 성능과 비용을 결정하는 핵심 아키텍처 설계**이며, 이를 올바르게 구현해야 실제 서비스에서 빠르고 효율적인 검색 시스템을 구축할 수 있습니다.

따라하기 — FAISS 인덱스 만들기

이번 실습에서는 지금까지 배운 내용을 실제 코드로 구현합니다.

최종 목표는 다음과 같습니다.

1. 여러 개의 PDF 문서를 읽는다.
2. 문서를 작은 청크(Chunk)로 분할한다.
3. 각 청크를 임베딩(Embedding)하여 벡터(Vector)로 변환한다.
4. 생성된 벡터를 이용하여 FAISS 인덱스를 생성한다.
5. 생성된 인덱스에서 `similarity_search()`를 이용해 의미 기반 검색을 수행한다.

0. 실습 준비(설치)

먼저 실습 환경을 준비합니다.

```
conda activate agentic
```

사용하는 라이브러리를 설치합니다.

```
pip install faiss-cpu langchain-community langchain-google-genai pypdf
# 만약 Chroma 도 함께 실습하려면 다음 패키지도 설치합니다.
pip install langchain-chroma
```

각 패키지의 역할

① faiss-cpu

FAISS 벡터 검색 엔진(CPU 버전)입니다.

벡터를 저장하고 매우 빠르게 검색하는 기능을 제공합니다.

② langchain-community

LangChain 에서 제공하는 다양한 커뮤니티 기능을 사용할 수 있습니다.

이번 실습에서는

- FAISS
- PyPDFLoader

등을 사용하기 위해 필요합니다.

③ langchain-google-genai

Google Gemini 임베딩 모델을 사용하기 위한 라이브러리입니다.

문장을 숫자 벡터로 변환하는 기능을 제공합니다.

④ pypdf

PDF 파일을 읽기 위한 라이브러리입니다.

PDF 내부의 텍스트를 추출합니다.

실습에 사용할 PDF

다음 두 개의 PDF 를 사용합니다.

- 제품매뉴얼_로봇청소기.pdf
- 제품매뉴얼_스마트워치.pdf

그리고 임베딩을 수행하기 위해서는 GOOGLE_API_KEY 환경 변수가 필요합니다.

1. 여러 PDF 로드 → 청킹

로드·청킹을 여러 PDF 에 적용합니다.

```
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parent))
from common import get_embeddings, DOCS, DATA
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

def load_and_chunk(filenamees):
    """여러 PDF를 로드·청킹해 청크 리스트 반환."""
    docs = []
    for name in filenamees:
        path = DOCS / name
        if not path.exists():
            raise SystemExit(f"[파일 없음] {path}")
        docs.extend(PyPDFLoader(str(path)).load()) # 여러 PDF를 한 리스트로 합침
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
    chunks = splitter.split_documents(docs) # 13강과 동일한 청킹
    print(f"문서 페이지: {len(docs)} / 청크: {len(chunks)}")
    return chunks
```

여러 PDF 를 한 리스트로 합쳐 청킹합니다. 로봇청소기·스마트워치 매뉴얼이 **하나의 인덱스**에 들어가는 거죠.

2. FAISS 인덱스 만들기

FAISS.from_documents 로 청크를 임베딩하고 인덱스를 만듭니다.

```
from langchain_community.vectorstores import FAISS

emb = get_embeddings()
chunks = load_and_chunk(["제품매뉴얼_로봇청소기.pdf", "제품매뉴얼_스마트워치.pdf"])

# 청크 임베딩 + FAISS 인덱스 적재 (여기서 임베딩 API 비용 발생!)
vs = FAISS.from_documents(chunks, emb)
```

from_documents 한 줄 안에서 두 가지가 일어납니다.

1. 모든 청크를 **임베딩**(14 강의 embed_documents)
2. 그 벡터들로 **FAISS 인덱스 생성**

비용 주의: 이 한 줄이 모든 청크를 임베딩하므로 **API 비용이 발생합니다**. 그래서 인덱싱은 한번만 하고 저장해 두는 거죠

3. similarity_search()로 검색

만든 인덱스에서 의미 검색을 합니다.

```
query = "로봇청소기 물걸레 되나요?"
results = vs.similarity_search(query, k=3) # 가장 가까운 청크 3개

print(f"[검색] {query}")
for i, d in enumerate(results, 1):
    src = d.metadata.get("source", "?").split("/")[-1]
    print(f"\n[{i}] (출처: {src} p.{d.metadata.get('page')})")
    print(d.page_content[:150], "...")
```

예상 출력

[검색] 로봇청소기 물걸레 되나요?

[1] (출처: 제품매뉴얼_로봇청소기.pdf p.1)

물걸레 기능: 본 제품은 흡입과 물걸레 청소를 동시에 지원합니다. 물탱크에 ...

[2] (출처: 제품매뉴얼_로봇청소기.pdf p.1)

청소 모드 ...

similarity_search(query, k=3)는:

1. 질문을 임베딩하고
2. 인덱스에서 가장 가까운 청크 3 개를 ANN 으로 빠르게 찾습니다.

numpy 검색과 결과는 같지만, **훨씬 빠릅니다**(ANN).

4. 의미 검색의 위력 확인

흥미로운 점: 스마트워치 매뉴얼도 같이 인덱싱했는데, "물걸레" 질문에는 로봇청소기 청크만 상위로 올라왔습니다.

인덱스에 든 것: 로봇청소기 청크 + 스마트워치 청크 (섞여 있음)
"물걸레" 질문 → 로봇청소기 청크가 의미상 가까움 → 상위 결과

스마트워치엔 "물걸레"가 없으니(의미가 멀어서) 안 나오고, 로봇청소기 청크가 나옵니다. 여러 문서를 한 인덱스에 넣어도, 의미로 알맞은 것만 찾아 주는 거죠. 이게 의미 기반 검색의 위력입니다.

5. metadata 로 출처 추적

검색 결과에는 문서 내용뿐 아니라 메타데이터도 함께 저장됩니다.

```
src = d.metadata.get("source")
page = d.metadata.get("page")
```

에 들어있는 정보

예를 들어

source

↓

제품매뉴얼_로봇청소기.pdf

page

↓

1

처럼 어느 파일의 몇 페이지에서 나온 내용인지 확인할 수 있습니다.

따라서 다음과 같이 출처를 표시할 수 있습니다.

이 답변은 제품매뉴얼_로봇청소기.pdf 1 페이지에서 가져왔습니다.

핵심 정리

이번 실습에서 반드시 기억해야 할 내용은 다음과 같습니다.

1. faiss-cpu, langchain-community, langchain-google-genai, pypdf 를 설치하여 FAISS 기반 벡터 검색 환경을 준비합니다.
2. 여러 PDF 를 PyPDFLoader 로 읽은 후 하나의 리스트로 합치고, RecursiveCharacterTextSplitter 를 사용하여 청크로 분할합니다.
3. FAISS.from_documents(chunks, emb)는 모든 청크를 임베딩하고 FAISS 인덱스를 생성하는 작업을 한 번에 수행하며, 이 과정에서 임베딩 API 사용 비용이 발생합니다.
4. similarity_search(query, k=3)는 질문을 임베딩한 뒤 ANN 알고리즘을 이용하여 가장 의미적으로 가까운 청크를 빠르게 검색합니다.
5. 여러 문서를 하나의 인덱스에 저장해도 질문의 의미와 가장 가까운 문서만 검색 결과의 상위에 나타나는 것이 의미 기반 검색(Semantic Search)의 핵심 특징입니다.
6. 검색 결과에는 metadata 가 함께 포함되어 있어 출처 파일과 페이지를 추적할 수 있으며, 이후 RAG 시스템에서 답변의 출처를 사용자에게 제공하는 데 활용됩니다.

따라하기 — 인덱스 저장과 재로드

목표

만든 FAISS 인덱스를 `save_local` 로 디스크에 저장하고, `load_local` 로 재임베딩 없이 다시 불러옵니다. 이게 04번에서 배운 "인덱싱/서비스 분리"의 실제 구현입니다.

1. 인덱스 저장 — `save_local`

만든 인덱스를 파일로 저장합니다.

```
from common import DATA
INDEX_DIR = str(DATA / "faiss_index") # 인덱스를 저장할 폴더

# [왜] 매 요청마다 PDF를 다시 임베딩하면 비용·지연이 폭발한다.
#     비싼 계산은 미리 해서 파일로 저장해 둔다.
vs.save_local(INDEX_DIR)
print("인덱스 저장 완료 →", INDEX_DIR)
```

`save_local` 은 지정한 폴더(`faiss_index/`)에 두 파일을 만듭니다.

파일	내용
<code>index.faiss</code>	벡터 인덱스(검색 구조)
<code>index.pkl</code>	청크 텍스트·메타데이터

이제 이 폴더만 있으면 임베딩을 다시 안 해도 검색할 수 있습니다.

2. 인덱스 재로드 — load_local

저장한 인덱스를 다시 불러옵니다.

```
vs2 = FAISS.load_local(
    INDEX_DIR, emb,
    allow_dangerous_deserialization=True, # pickle 역직렬화 허용
)
print("재로드 후 검색:", len(vs2.similarity_search("배터리 충전", k=2)), "건")
```

핵심: `load_local` 은 저장된 벡터를 그대로 읽습니다. 임베딩 API를 다시 호출하지 않습니다. 그래서 빠르고 공짜죠. 04번에서 배운 "서버 기동 시 1회 로드"가 바로 이것입니다.

```
from_documents : 임베딩 API 호출 0 (비쌈) — 인덱싱 때 한 번
load_local     : 임베딩 API 호출 X (공짜) — 서비스 때 매번 가능
```

3. allow_dangerous_deserialization 주의

`load_local` 에는 `allow_dangerous_deserialization=True` 가 필요합니다. 이름이 무섭죠? 이유가 있습니다.

```
allow_dangerous_deserialization=True # pickle 역직렬화 허용
```

FAISS 인덱스의 `index.pkl` 은 파이썬 **pickle** 형식입니다. pickle은 역직렬화(읽기) 과정에서 임의 코드가 실행될 수 있어, 악의적으로 조작된 pickle 파일은 위험합니다.

반드시 지킬 것: 이 옵션은 내가 직접 만든 신뢰된 인덱스 파일에만 사용하세요. 외부에서 받은 인덱스 파일은 절대 로드하지 마세요. 누군가 조작한 pickle을 로드하면 내 시스템에서 악성 코드가 돌 수 있습니다(10강 보안 정신).

4. 인덱싱과 서비스 분리 실습

04번의 아키텍처를 코드로 정리하면:

```
# == build_index.py (배치 — 한 번만) ==
chunks = load_and_chunk(["제품매뉴얼_로봇청소기.pdf", "제품매뉴얼_스마트워치.pdf"])
vs = FAISS.from_documents(chunks, emb) # 임베딩 (비쌈)
vs.save_local("faiss_index")          # 저장
# → 끝. 다시 실행할 필요 없음 (문서 바뀌기 전까지)

# == service.py (서버 — 기동 시 1회 로드, 요청마다 검색) ==
vs = FAISS.load_local("faiss_index", emb, allow_dangerous_deserialization=True) # 1회
def search(question):
    return vs.similarity_search(question, k=3) # 요청마다 (빠름, 공짜)
```

`build_index.py` 는 배포할 때 한 번 돌리고, `service.py` 는 서버가 떠 있는 동안 검색만 합니다. 임베딩 비용은 인덱싱 때 한 번만 들죠.

5. 점수와 함께 검색 (참고)

검색 결과의 "유사도 점수"도 볼 수 있습니다.

```
scored = vs2.similarity_search_with_score(query, k=2)
print("[점수와 함께] (FAISS 점수는 작을수록 가까움)")
for d, score in scored:
    print(f"거리 {score:.3f} | {d.page_content[:50]} ...")
```

예상 출력

```
[점수와 함께]
거리 0.245 | 물걸레 기능: 본 제품은 흡입과 물걸레 청소를 ...
거리 0.398 | 청소 모드는 표준/강력/조용 모드를 ...
```

03번에서 강조했듯, **FAISS** 점수는 "거리"라 작을수록 가깝습니다. 0.245가 0.398보다 더 관련성 높은 청크죠. 점수로 "검색 결과가 얼마나 믿을 만한지"를 가늠할 수 있습니다(16강에서 활용).

핵심 정리

이번 챕터에서 반드시 기억해야 할 내용은 다음과 같습니다.

1. `save_local(폴더)`를 사용하면 생성한 FAISS 인덱스를 **`index.faiss`와 `index.pkl` 파일 형태로 디스크에 저장**할 수 있습니다.
2. `load_local(폴더, emb, ...)`는 **저장된 벡터를 그대로 메모리로 불러오기 때문에 임베딩 API 를 다시 호출하지 않으며**, 빠르고 추가 비용이 발생하지 않습니다.
3. `allow_dangerous_deserialization=True` 는 **Python 의 pickle 파일을 역직렬화하기 위한 옵션**이며, **직접 생성한 신뢰할 수 있는 인덱스 파일에만 사용**해야 하고 외부에서 받은 파일에는 사용해서는 안 됩니다.
4. 실제 서비스에서는 **`build_index.py` 에서 문서를 임베딩하고 인덱스를 생성·저장한 뒤, `service.py`에서는 `load_local()`로 인덱스를 한 번만 불러와 검색만 수행하도록 설계**해야 합니다.
5. `similarity_search_with_score()`를 사용하면 검색 결과와 함께 **거리(distance) 점수**를 확인할 수 있으며, **FAISS에서는 점수가 작을수록 질문과 더 유사한 문서를** 의미합니다.
6. 이러한 **인덱스 저장과 재로드 구조**를 사용하면 임베딩 비용을 한 번만 지불하고도 반복적으로 빠른 의미 기반 검색을 수행할 수 있으며, 이것이 RAG 시스템의 효율성과 성능을 높이는 핵심 원리입니다.

5. Vector DB 를 활용한 검색

메타데이터 필터링 검색

여러 개의 PDF 문서를 하나의 벡터 DB 인덱스에 저장하고, 전체 문서를 대상으로 의미 기반 검색(Semantic Search)을 수행했습니다.

하지만 실제 서비스에서는 항상 전체 문서를 검색하는 것이 좋은 것은 아닙니다.

예를 들어,

- 회원 등급에 대한 질문은 **멤버십 정책 문서**에서만 검색하고 싶을 수 있습니다.
- 환불에 대한 질문은 **환불 정책 문서**에서만 검색하고 싶을 수 있습니다.
- 연차 휴가에 대한 질문은 **직원 핸드북**에서만 검색하고 싶을 수 있습니다.

이처럼 **특정 문서나 특정 조건으로 검색 범위를 제한하는 기능을 메타데이터 필터링 검색**이라고 합니다.

여러 PDF 를 하나의 인덱스에 저장하더라도, 각 청크(Chunk)의 **metadata** 에 출처(**source**)를 **함께 저장하면** `filter={"source": "멤버십정책.pdf"}` 와 같이 **특정 문서 안에서만** 검색할 수 있습니다.

이 기능은 **Chroma** 에서 강력하게 지원되며, FAISS 는 메타데이터 필터 기능이 제한적이기 때문에 이 실습에서는 Chroma 를 사용합니다.

1. 왜 필터가 필요한가

여러 문서를 한 인덱스에 넣으면 편하지만, 가끔 검색 범위를 좁히고 싶을 때가 있습니다.

질문: "VIP 등급이 받는 혜택은?"

- 전체 검색: 환불정책 · 핸드북 · 멤버십정책 청크가 다 후보
- 엉뚱한 문서(환불정책)의 청크가 섞여 들어올 수 있음!

원하는 것: "멤버십정책.pdf 안에서만" 검색

"멤버십 혜택"은 멤버십 문서에만 있는데, 전체에서 검색하면 다른 문서의 비슷한 표현이 섞일 수 있습니다. 특정 문서로 범위를 좁히면 노이즈가 줄어 정확해지죠.

2. metadata에 출처 보존

필터링하려면 먼저 각 청크에 무엇으로 필터할지(출처)를 metadata에 넣어야 합니다. (실습 파일:

`sections/15_vector_database/07_메타데이터-필터링-검색.py`)

```
def load_and_chunk(filenamees):
    """각 청크 metadata에 source(파일명)를 보존한다."""
    chunks = []
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
    for name in filenamees:
        docs = PyPDFLoader(str(DOCS / name)).load()
        parts = splitter.split_documents(docs)
        # PyPDFLoader의 source는 전체 경로 → 필터 키로 쓰기 좋게 '파일명'으로 정규화
        for d in parts:
            d.metadata["source"] = name # 예: "멤버십정책.pdf"
        chunks.extend(parts)
    return chunks
```

`PyPDFLoader`가 넣어 주는 `source`는 전체 경로(`/Users/.../멤버십정책.pdf`)라 필터 키로 불편합니다. 그래서 파일명만(`멤버십정책.pdf`)으로 정규화해 두죠. 이러면 `filter={"source": "멤버십정책.pdf"}`로 깔끔하게 필터할 수 있습니다.

3. Chroma로 인덱싱

메타데이터 필터는 **Chroma**가 강합니다(03번). Chroma로 인덱스를 만듭니다.

```
from langchain_community.vectorstores import Chroma

chunks = load_and_chunk(["환불교환정책.pdf", "멤버십정책.pdf", "직원핸드북.pdf"])
vs = Chroma.from_documents(chunks, get_embeddings(), persist_directory=CHROMA_DIR)
```

왜 FAISS 말고 Chroma? FAISS도 검색은 되지만 "source가 X인 청크만" 같은 정확한 필터가 약합니다. Chroma는 `where` 필터(metadata 조건)를 네이티브로 지원해, 필터링이 정확하고 편합니다.

4. 필터로 범위 좁히기

`similarity_search`에 `filter`를 주면 그 조건의 청크만 검색합니다.

```
def search_in_doc(vs, query, source=None, k=3):
    """source 지정 시 해당 문서 안에서만, 미지정 시 전체에서 검색한다."""
    flt = {"source": source} if source else None
    return vs.similarity_search(query, k=k, filter=flt)

# ① 필터 없이 전체 검색 — 엉뚱한 문서가 섞일 수 있음
search_in_doc(vs, "VIP 등급 혜택은?")

# ② 멤버십정책.pdf 안에서만 — 정확히 그 문서 청크만 후보
search_in_doc(vs, "VIP 등급 혜택은?", source="멤버십정책.pdf")
```

예상 출력

```
[필터 없음 — 전체 검색]
[1] (출처: 멤버십정책.pdf) VIP 등급은 연 100만원 이상 구매 시 ...
[2] (출처: 환불교환정책.pdf) ... ← 엉뚱한 문서가 섞임!

[필터: 멤버십정책.pdf 안에서만]
[1] (출처: 멤버십정책.pdf) VIP 등급은 연 100만원 이상 구매 시 ...
[2] (출처: 멤버십정책.pdf) 등급별 적립률은 ... ← 모두 멤버십 문서!
```

필터를 주니 멤버십 문서 청크만 결과에 나옵니다. 노이즈가 사라졌죠.

5. 필터를 라우팅처럼 활용

필터로 검색 범위를 바꾸면, 질문 종류에 따라 알맞은 문서로 라우팅하는 효과를 낼 수 있습니다.

```
# "연차 휴가" 질문 → 직원핸드북에서만
search_in_doc(vs, "연차 휴가는 며칠인가요?", source="직원핸드북.pdf")

# "환불" 질문 → 환불정책에서만
search_in_doc(vs, "환불 며칠 걸려요?", source="환불교환정책.pdf")
```

질문 주제에 맞는 문서로 필터하면, 그 문서 안에서만 정확히 찾습니다. 12강의 도구 라우팅과 비슷한 발상이죠 — "이 질문엔 이 문서". (어느 문서로 필터할지는 LLM이 정하게 할 수도 있습니다.)

6. 필터 + LLM 답변

필터로 노이즈를 줄인 뒤 LLM에게 답을 시키면 더 정확합니다.

```
docs = search_in_doc(vs, "VIP 등급 혜택은?", source="멤버십정책.pdf", k=3)
context = "\n\n".join(d.page_content for d in docs)
llm = get_chat(temperature=0.0)
answer = llm.invoke(
    f"아래 [문서]만 근거로 한국어로 답하라.\n\n[문서]\n{context}\n\n[질문] VIP 등급 혜택은?"
).content
print(answer)
```

필터로 관련 문서만 추린 뒤 그 내용을 근거로 답하니, 환각이 줄고 정확해집니다. 이게 RAG QA의 미리보기입니다.

핵심 정리

이번 내용에서 반드시 기억해야 할 내용은 다음과 같습니다.

1. 여러 문서를 하나의 벡터 인덱스에 저장하더라도 **각 청크의 metadata에 출처(source)를 저장하면 특정 문서만 대상으로 검색할 수 있습니다.**
2. PyPDFLoader가 저장하는 전체 경로 대신 **파일명만 source에 저장하면 필터 조건으로 사용하기 편리합니다.**
3. **Chroma는 메타데이터 기반 필터링(filter={"source": "..."})을 기본적으로 지원하므로 특정 문서, 특정 범위, 특정 조건만 검색하는 데 적합합니다.**
4. 필터를 사용하면 **검색 범위를 줄여 노이즈를 제거하고 검색 정확도를 높일 수 있으며, 질문의 종류에 따라 적절한 문서를 선택하는 라우팅(Routing) 방식으로 활용**할 수 있습니다.
5. 필터링된 검색 결과만 LLM에게 전달하면 **관련 없는 문서가 제외되어 환각(Hallucination)을 줄이고 더 정확한 RAG 답변을 생성**할 수 있습니다.

인덱스 증분 업데이트

새 문서가 추가되었다고 해서 **기존 모든 문서를 다시 임베딩할 필요는 없습니다.**

대신 `add_documents()`를 이용하면 새로운 문서의 청크만 임베딩하여 기존 인덱스에 추가할 수 있습니다.

즉, 기존 벡터는 그대로 유지하고, 새 벡터만 추가하기 때문에 시간과 비용을 크게 절약할 수 있습니다.

1. 문서가 추가되면?

인덱스를 만든 뒤 새 문서가 생기는 건 흔한 일입니다.

처음: 환불정책.pdf로 인덱스 생성
나중: 멤버십정책.pdf가 새로 추가됨
→ 멤버십 질문에 답하려면 인덱스에 멤버십 지식이 필요!

이때 두 가지 방법이 있습니다.

- **전체 재빌드**: 환불+멤버십 전체를 다시 임베딩 → 간단하지만 기존 것도 다시 임베딩(낭비)
- **증분 업데이트**: 멤버십만 임베딩해 기존 인덱스에 덧붙임 → 효율적

문서가 많으면 전체 재빌드는 낭비가 큼니다. 이미 임베딩한 수천 개를 또 임베딩하니까요. 증분 업데이트가 답입니다.

2. add_documents — 덧붙이기

`add_documents`로 새 청크만 기존 인덱스에 추가합니다. (실습 파일: `sections/15_vector_database/08_인덱스-증분-업데이트.py`)

```
# 1단계: 환불정책으로 최초 인덱싱 + 저장
base_chunks = chunk_pdf("환불교환정책.pdf")
vs = FAISS.from_documents(base_chunks, emb) # 환불 청크만 임베딩
vs.save_local(INDEX_DIR)

# 2단계: 나중에 멤버십정책만 증분 추가 (서버 재시작 가정)
vs2 = FAISS.load_local(INDEX_DIR, emb, allow_dangerous_deserialization=True) # 기존 로드
new_chunks = chunk_pdf("멤버십정책.pdf")
vs2.add_documents(new_chunks) # 새 청크만 임베딩해 덧붙임 (기존은 재계산 X!)
vs2.save_local(INDEX_DIR) # 갱신된 인덱스 재저장
```

핵심: `add_documents(new_chunks)`는 새 청크만 임베딩합니다. 기존 환불 청크는 이미 인덱스에 있으니 다시 임베딩하지 않죠. 멤버십 청크만 추가 비용이 듭니다.

3. 증분 전후 비교

증분 추가 전후로 같은 질문을 검색해 봅니다.

```
q = "VIP 등급 조건은?"

# 증분 전 (환불정책만 있을 때) — 멤버십 지식 없음
before = vs.similarity_search(q, k=2)
# → 환불 관련 청크만 나옴 (멤버십 답을 못 찾음)

# 증분 후 (멤버십 추가됨) — 멤버십 지식 검색됨
after = vs2.similarity_search(q, k=2)
# → 멤버십정책 청크가 나옴!
```

예상 출력

```
[증분 전 검색] VIP 등급 조건은?
[1] 환불은 상품 수령 후 7일 이내 ... ← 멤버십 지식 없어 엉뚱한 결과

[증분 후 검색] VIP 등급 조건은?
[1] (출처: 멤버십정책.pdf) VIP 등급은 연 100만원 이상 구매 시 ... ← 새 지식 검색됨!
```

증분 추가 후, "VIP 등급" 질문에 **멤버십 문서**가 나옵니다. 새 지식이 인덱스에 반영된 거죠.

4. 기존 지식은 유지된다

증분 추가는 기존 벡터를 **덮어쓰지 않고** 누적합니다. 그래서 기존 지식도 그대로 검색됩니다.

```
# 멤버십을 추가한 뒤에도 환불 지식은 그대로
keep = vs2.similarity_search("환불은 며칠 걸리나요?", k=2)
print("환불 관련 검색:", len(keep), "건") # 여전히 검색됨!
```

멤버십을 추가했다고 환불 지식이 사라지지 않습니다. 누적이니까요. 환불 질문엔 환불 청크가, 멤버십 질문엔 멤버십 청크가 나옵니다. 둘 다 인덱스에 살아 있죠.

5. 증분 vs 재빌드, 언제?

상황	권장	이유
새 문서 추가	증분 (<code>add_documents</code>)	기존 재임베딩 안 함 → 효율
기존 문서 대량 수정	전체 재빌드	수정된 청크를 일일이 찾기 복잡
문서 삭제	상황에 따라	FAISS는 삭제가 까다로움

문서 추가는 증분이 효율적입니다. 하지만 기존 문서가 많이 바뀌거나 삭제되면, 어느 벡터를 지울지 추적하기 복잡해서 전체 재빌드가 더 간단할 수 있습니다. 상황에 맞게 고릅니다.

실무 팁: 문서가 자주 바뀌고 삭제·수정이 많으면, FAISS보다 **Chroma**나 클라우드 벡터DB가 갱신 관리에 더 편합니다. FAISS는 "주로 추가, 가끔 전체 재빌드"에 적합하죠.

핵심 정리

이번 내용에서 반드시 기억해야 할 내용은 다음과 같습니다.

1. 새로운 문서가 추가되었을 때는 기존 인덱스를 모두 다시 만드는 대신 **`add_documents()`**를 사용하여 새 문서만 추가할 수 있습니다.
2. **`add_documents()`**는 새로운 청크만 임베딩하고 기존 벡터는 다시 계산하지 않으므로, API 비용과 처리 시간을 크게 줄일 수 있습니다.
3. 증분 업데이트를 수행하면 새로운 문서의 지식은 인덱스에 추가되고, 기존 문서의 지식도 그대로 유지되므로 하나의 인덱스에서 모두 검색할 수 있습니다.
4. 문서 추가에는 증분 업데이트가 가장 효율적이지만, 기존 문서의 대량 수정이나 삭제가 발생한 경우에는 전체 재빌드가 더 단순하고 관리하기 쉬운 방법이 될 수 있습니다.
5. 문서의 추가·수정·삭제가 매우 자주 발생하는 환경에서는 **Chroma**나 클라우드 기반 벡터 DB가 관리 측면에서 더 유리하며, FAISS는 문서 추가가 주를 이루고 가끔 전체 재빌드를 수행하는 환경에 적합합니다.
6. 증분 업데이트는 RAG 시스템을 운영할 때 새로운 지식을 빠르게 반영하면서도 기존 임베딩을 재사용하여 비용과 성능을 최적화하는 핵심 기술입니다.